

Instituto Tecnológico de Culiacán



Ingeniería en Sistemas Computacionales

Tópicos de inteligencia artificial.

Maestro: Zuriel Mora.

Nombre:

Peñuelas López Luis Antonio

Peraza Medina Eliezer Daniel

Horario: 12 - 13

<https://github.com/InteligenciaArtificial-Equipo/Topicos-Inteligencia-Artificial/tree/78828cecaa2ccb92168bec9d2450d0847cb2f903/Unidad-3/Tarea%20II%20Bonus%20Modulo%20III>

Introducción

El **Problema del Vendedor Ambulante (TSP, por sus siglas en inglés)** es un problema clásico en el campo de la **optimización combinatoria**. Consiste en determinar la ruta más corta posible que permita a un viajero visitar todas las ciudades de una lista exactamente una vez y regresar al punto de partida.

Este problema tiene una gran relevancia práctica, ya que representa de forma abstracta muchas situaciones reales como la **planificación de rutas de transporte, logística de entregas, diseño de circuitos eléctricos y organización de itinerarios**.

El desafío principal del TSP radica en que el número de rutas posibles crece de manera factorial con el número de ciudades, lo que hace que sea **computacionalmente imposible resolverlo por fuerza bruta** cuando hay muchas ciudades. Por ello, se utilizan **metaheurísticas** como los **algoritmos genéticos**, que ofrecen soluciones aproximadas de alta calidad en tiempos razonables.

Descripción del enfoque mediante Algoritmos Genéticos

Un **algoritmo genético (AG)** es un método de búsqueda inspirado en la evolución biológica. Utiliza conceptos como **selección natural, cruce (reproducción)** y **mutación** para mejorar iterativamente una población de soluciones candidatas.

En este trabajo, se diseñó un algoritmo genético para resolver el TSP mediante los siguientes pasos:

1. Representación:

Cada individuo de la población representa una **ruta**, es decir, un orden de visita de las ciudades (una permutación de sus índices).

2. Población

inicial:

Se genera aleatoriamente un conjunto de rutas iniciales (por ejemplo, 150 rutas distintas).

3. Función

de

aptitud

(fitness):

Se define como el **inverso de la distancia total** recorrida. Cuanto menor sea la distancia, mayor será la aptitud del individuo.

4. **Selección:**

Se aplica el método de **torneo**, donde varios individuos compiten y el más apto pasa a la siguiente fase.

5. **Cruce**

(crossover):

Se combina información genética de dos padres mediante un **cruce ordenado (OX)**, que intercambia segmentos de las rutas sin repetir ciudades.

6. **Mutación:**

Se aplica una **mutación por intercambio**, que cambia el orden de dos ciudades con una pequeña probabilidad (tasa de mutación $\approx 12\%$), manteniendo la diversidad genética.

7. **Elitismo:**

Se conserva el mejor individuo de cada generación para evitar que se pierdan buenas soluciones.

8. **Iteraciones:**

El proceso se repite durante un número fijo de generaciones (por ejemplo, 400), buscando mejorar progresivamente la calidad de las rutas.

Resultados del algoritmo.

Tras ejecutar el algoritmo con una población de 150 individuos durante 400 generaciones, se obtuvo la siguiente ruta óptima aproximada:

Mejor ruta encontrada:

G -> H -> F -> E -> A -> C -> B -> D -> G

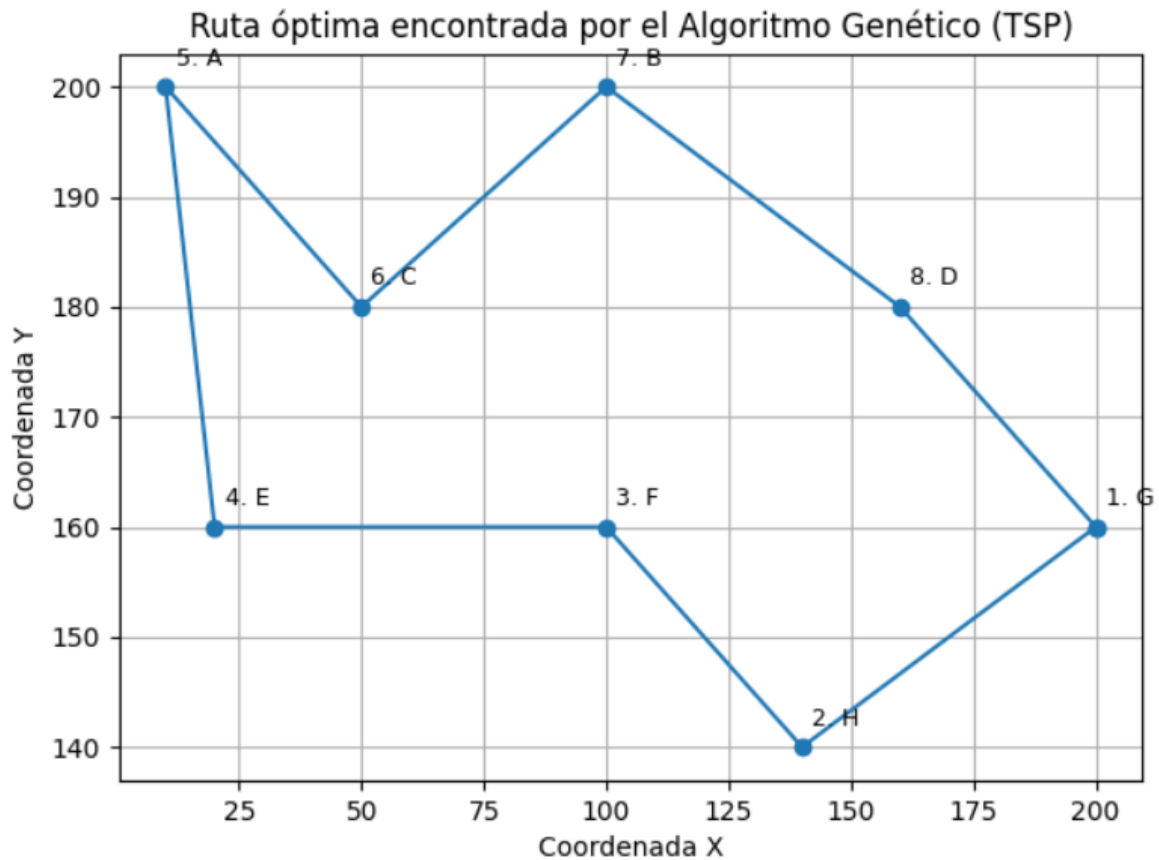
Distancia total: 435.738

El algoritmo fue capaz de reducir significativamente la distancia total respecto a la población inicial (que superaba los 500 puntos de distancia promedio).

La convergencia fue estable después de unas 200 generaciones, mostrando la eficiencia del método.

Visualización de la ruta:

El gráfico generado muestra el recorrido del viajero entre las ciudades en un plano bidimensional, con la ruta óptima conectando todos los puntos y regresando al origen.



Desafíos enfrentados y soluciones implementadas

Durante el desarrollo de la tarea se presentaron varios desafíos:

- **Problemas de ejecución en entorno local:**
Inicialmente, Python en Windows no reconocía la biblioteca matplotlib ni la instalación de pip.
Solución: Se migró la ejecución del código a **Google Colab**, que ya cuenta con todas las dependencias instaladas.
- **Manejo de duplicados en el cruce genético:**
Algunos métodos de cruce podían generar rutas inválidas (con ciudades repetidas).
Solución: Se implementó un **cruce ordenado (OX)** que garantiza rutas válidas.
- **Convergencia prematura:**
En ciertas ejecuciones el algoritmo se estancaba.
Solución: Se ajustó la **tasa de mutación (0.12)** y se aplicó **elitismo**, mejorando la estabilidad de la evolución.

Conclusión

El uso de algoritmos genéticos resulta altamente efectivo para resolver el problema del vendedor ambulante, especialmente cuando se busca una solución aproximada sin necesidad de evaluar todas las combinaciones posibles.

El enfoque implementado permitió obtener rutas óptimas o casi óptimas en pocas generaciones, mostrando cómo la combinación de selección, cruce, mutación y elitismo logra un equilibrio entre exploración y explotación del espacio de soluciones.

El trabajo realizado demuestra la potencia de las técnicas evolutivas en problemas de optimización complejos y abre la puerta a mejoras como la hibridación con otros métodos metaheurísticos o el ajuste adaptativo de parámetros.

Apéndice: Código fuente completo comentado.

```
# =====  
  
# PROBLEMA DEL VIAJERO (TSP) RESUELTO CON ALGORITMO GENÉTICO  
  
# =====  
  
import random  
  
import math  
  
import matplotlib.pyplot as plt  
  
  
# Fijamos una semilla para reproducibilidad (los mismos resultados cada ejecución)  
random.seed(42)  
  
  
# 1. DEFINICIÓN DE LAS CIUDADES Y SUS COORDENADAS  
  
  
ciudades = {  
    'A': (10, 200),  
    'B': (100, 200),  
    'C': (50, 180),  
}
```

```
'D': (160, 180),  
'E': (20, 160),  
'F': (100, 160),  
'G': (200, 160),  
'H': (140, 140)  
}
```

```
# Obtenemos las listas de nombres y coordenadas
```

```
nombres = list(ciudades.keys())
```

```
coords = [ciudades[n] for n in nombres]
```

```
# 2. FUNCIONES AUXILIARES DE DISTANCIA Y FITNESS
```

```
def distancia(a, b):
```

```
    #Calcula la distancia euclidiana entre dos puntos.
```

```
    return math.hypot(a[0] - b[0], a[1] - b[1])
```

```
def distancia_total(ruta):
```

```
    #Suma la distancia total recorrida en una ruta cerrada.
```

```
    total = 0
```

```
    for i in range(len(ruta)):
```

```
        a = coords[ruta[i]]
```

```
        b = coords[ruta[(i + 1) % len(ruta)]] # % para volver al inicio
```

```
        total += distancia(a, b)
```

```
    return total
```

```
def fitness(individuo):
```

```
    #Evalúa qué tan buena es una ruta.
```

```
    #Cuanto menor sea la distancia, mayor será el fitness.
```

```
    return 1 / (distancia_total(individuo) + 1e-9)
```

3. CREACIÓN DE LA POBLACIÓN INICIAL

```
def crear_poblacion(tamaño, num_ciudades):
```

```
    #Crea la población inicial aleatoria.
```

```
    #Cada individuo es una permutación de los índices de las ciudades.
```

```
    base = list(range(num_ciudades))
```

```
    return [random.sample(base, len(base)) for _ in range(tamaño)]
```

4. OPERADORES GENÉTICOS (SELECCIÓN, CRUCE Y MUTACIÓN)

```
def seleccion_torneo(poblacion, k=5):
```

```
    #SELECCIÓN POR TORNEO:
```

```
    #- Escoge aleatoriamente k individuos de la población.
```

```
    #- De esos k, el que tenga mejor fitness 'gana' y se selecciona como padre.
```

```
    candidatos = random.sample(poblacion, k)
```

```
return max(candidatos, key=fitness)
```

```
def crossover(p1, p2):
```

```
    #CRUCE ORDENADO (Ordered Crossover - OX):
```

```
    #- Selecciona un segmento del primer padre (p1).
```

```
    #- Rellena los huecos con las ciudades del segundo padre (p2),
```

```
    # respetando el orden en que aparecen y sin repetir ninguna ciudad.
```

```
    size = len(p1)
```

```
    a, b = sorted(random.sample(range(size), 2)) # puntos de corte
```

```
    hijo = [-1] * size # hijo vacío (usamos -1 como marcador)
```

```
    # Copiamos el segmento del primer padre
```

```
    hijo[a:b+1] = p1[a:b+1]
```

```
    # Rellenamos el resto con genes del segundo padre
```

```
    pos = (b + 1) % size
```

```
    p2pos = (b + 1) % size
```

```
    while -1 in hijo:
```

```
        gene = p2[p2pos]
```

```
        if gene not in hijo:
```

```
            hijo[pos] = gene
```

```
            pos = (pos + 1) % size
```

```
            p2pos = (p2pos + 1) % size
```

```
    return hijo
```



```
def mutacion_swap(individuo, tasa=0.12):
```

```
    #MUTACIÓN POR INTERCAMBIO:
```

```
    #- Con una cierta probabilidad (tasa),
```

```
    # intercambiamos dos ciudades de lugar dentro del individuo.
```

```
    #- Esto ayuda a mantener diversidad genética y evitar convergencia prematura.
```

```
    ind = individuo[:]
```

```
    if random.random() < tasa:
```

```
        i, j = random.sample(range(len(ind)), 2)
```

```
        ind[i], ind[j] = ind[j], ind[i]
```

```
    return ind
```

```
# -----
```

```
# 5. PROCESO DE EVOLUCIÓN (CICLO GENÉTICO)
```

```
# -----
```

```
def evolucionar(poblacion, elitismo=True):
```

```
    #Realiza una generación del algoritmo genético:
```

```
    #- Selecciona padres usando torneo.
```

```
    #- Aplica crossover y mutación para generar hijos.
```

```
    #- Conserva al mejor individuo si elitismo=True.
```

Elitismo: asegura que el mejor individuo no se pierda en la siguiente generación.

```
nueva = []
```

```
# Guardamos el mejor individuo (elitismo)
```

```
if elitismo:
```

```
    elite = max(poblacion, key=fitness)
```

```
    nueva.append(elite)
```

```
# Creamos el resto de la nueva generación
```

```
while len(nueva) < len(poblacion):
```

```
    # 1. Selección de padres
```

```
    p1 = seleccion_torneo(poblacion)
```

```
    p2 = seleccion_torneo(poblacion)
```

```
    # 2. Cruce
```

```
    hijo = crossover(p1, p2)
```

```
    # 3. Mutación
```

```
    hijo = mutacion_swap(hijo)
```

```
    # 4. Añadir hijo a la nueva población
```

```
    nueva.append(hijo)
```

```
# Retorna la nueva generación (misma cantidad de individuos)
```

```
return nueva[:len(poblacion)]
```

6. PARÁMETROS DEL ALGORITMO GENÉTICO

```
POBLACION = 150    # Tamaño de la población
GENERACIONES = 400  # Número de generaciones (iteraciones)
ciudades_totales = len(ciudades)
```

7. EJECUCIÓN DEL ALGORITMO GENÉTICO

```
poblacion = crear_poblacion(POBLACION, ciudades_totales)
hist_mejor = [] # Guardará la mejor distancia de cada generación

for gen in range(GENERACIONES):
    poblacion = evolucionar(poblacion) # Evolucionamos una generación
    mejor = min(poblacion, key=distancia_total) # Mejor individuo actual
    mejor_dist = distancia_total(mejor)
    hist_mejor.append(mejor_dist)

    # Mostramos progreso cada 100 generaciones
    if (gen + 1) % 100 == 0:
        print(f"Generación {gen+1}: mejor distancia = {mejor_dist:.3f}")
```

8. RESULTADOS FINALES

```
mejor_ruta = min(poblacion, key=distancia_total)
mejor_distancia = distancia_total(mejor_ruta)
```

```
mejor_ruta_nombres = [nombres[i] for i in mejor_ruta]
```

```
print("\nMejor ruta encontrada:")
```

```
print(" -> ".join(mejor_ruta_nombres) + f" -> {mejor_ruta_nombres[0]}")
```

```
print(f"Distancia total: {mejor_distancia:.3f}")
```

9. VISUALIZACIÓN DE LA MEJOR RUTA

```
# Preparamos coordenadas para graficar
```

```
xs = [coords[i][0] for i in mejor_ruta] + [coords[mejor_ruta[0]][0]]
```

```
ys = [coords[i][1] for i in mejor_ruta] + [coords[mejor_ruta[0]][1]]
```

```
# Dibujamos la ruta óptima encontrada
```

```
plt.figure(figsize=(7, 5))
```

```
plt.plot(xs, ys, marker='o', linestyle='-')
```

```
for i, idx in enumerate(mejor_ruta):
```

```
    plt.text(coords[idx][0]+2, coords[idx][1]+2, f"{i+1}. {nombres[idx]}", fontsize=9)
```

```
plt.title("Ruta óptima encontrada por el Algoritmo Genético (TSP)")
```

```
plt.xlabel("Coordenada X")
```

```
plt.ylabel("Coordenada Y")
```

```
plt.grid(True)
```

```
plt.show()
```